REQUIREMENTS MODELING AND UML DIAGRAMS

COMP6100001 - SOFTWARE ENGINEERING
SESSION 07

UNIVERSITAS BINA NUSANTARA

SUBJECT MATTER EXPERT
Maulin Nasari S.T., M.Kom.

SESSION 07 – REQUIREMENTS MODELING AND UML DIAGRAMS

LEARNING OBJECTIVES

At the end of this session, students will be expected to be able to:

- Understand the Purpose and Importance of System Modeling in Software Design
- Describe Key UML Diagram Types and Their Components
- Apply UML to Model Software Systems

SESSION 07 – REQUIREMENTS MODELING AND UML DIAGRAMS

CONTENT OUTLINE

- Requirements Analysis
- Scenario-Based Modeling
- Class-Based Modeling
- Functional Modeling
- Behavioral Modeling

INTRODUCTION TO REQUIREMENTS MODELING

Requirements modeling:

- Represents requirements using text and diagrams
- Makes requirements easier to:
  - Understand
  - Review
  - Validate

Why it matters:

- Enables early stakeholder feedback
- Helps detect errors before design
- Forms the basis for software design

HOW REQUIREMENTS MODELING IS DONE

Who builds the models:

- Software engineers / analysts
- Based on elicited stakeholder requirements

Modeling perspectives:

- Scenario-based modeling
- Class-based modeling
- Behavioral modeling

Work products:

- Usage scenarios (use cases)
- UML diagrams

Quality check:

- Correctness
- Completeness
- Consistency

REQUIREMENTS ANALYSIS

WHAT IS REQUIREMENTS ANALYSIS?

- Defines the software's **operational characteristics**
- Identifies:
  - Interfaces with other system elements
  - Constraints the software must satisfy
- Elaborates requirements from:
  - Inception
  - Elicitation
  - Negotiation

REQUIREMENTS ANALYSIS

PURPOSE OF REQUIREMENTS ANALYSIS

Requirements analysis aims to:

- Describe **what the customer needs**
- Provide a basis for **software design**
- Define requirements that can be **validated**

Outcome:

- Clear input for architecture, interface, and component design
- A reference for assessing quality after implementation

REQUIREMENTS ANALYSIS

REQUIREMENTS ANALYSIS PHILOSOPHY

“Analysis asks what, not how”

Focus on:

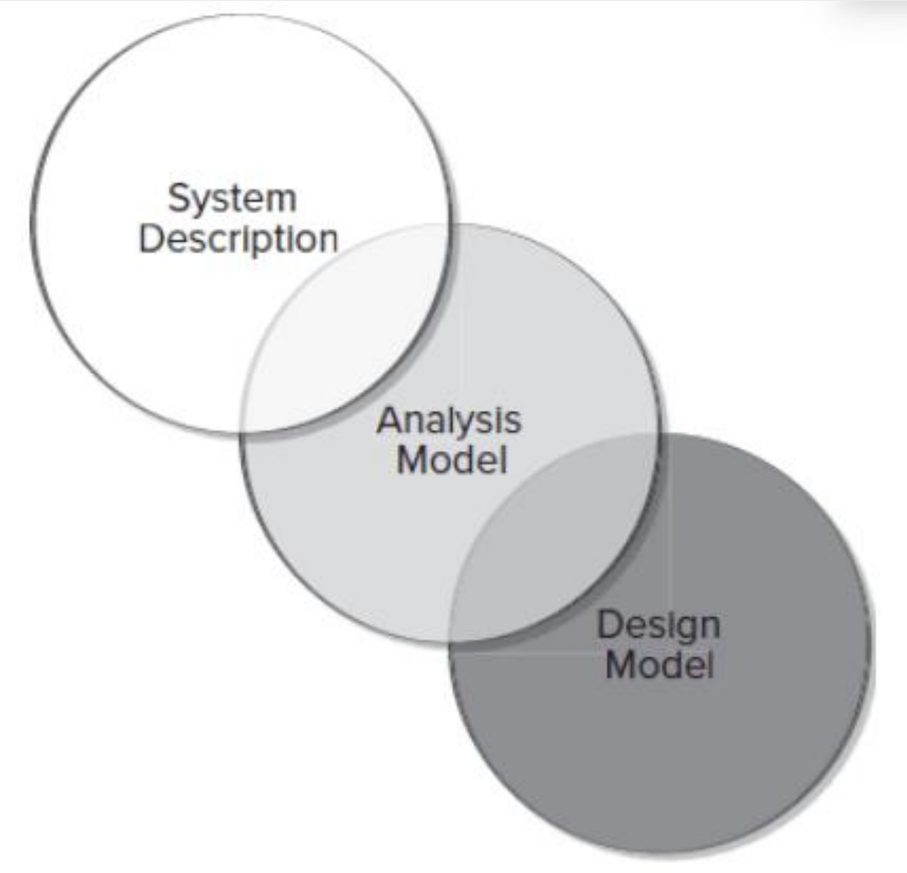
What interactions occur?
What data is manipulated?
What functions are required?
What behaviors are expected?
What constraints apply?

Not the focus:

How the system is implemented
Architectural or technical decisions

REQUIREMENTS ANALYSIS

ANALYSIS MODEL AS A BRIDGE



The analysis model:

- Bridges **system description** and **software design**
- Connects business needs to technical solutions
- Ensures traceability from requirements to design

Important note:

- Analysis and design may overlap
- Clear separation is not always possible

REQUIREMENTS ANALYSIS

REQUIREMENTS MODELING PRINCIPLE

Over time, many modeling methods have emerged, but they share **common principles**.
These principles guide:

- How we analyze problems
- How we build useful models
- How we avoid premature design

REQUIREMENTS ANALYSIS

REQUIREMENTS MODELING PRINCIPLE

Principle 1: Represent the Information Domain

- Identify:
  - Data entering the system
  - Data leaving the system
  - Data stored permanently
- Understand what information the system manages

Principle 2: Define Software Functions

- Identify functions that:
  - Provide value to users
  - Support internal processing
- Functions may:
  - Transform data
  - Control system behavior

Principle 3: Represent Software Behavior

- Software reacts to **external events**
- Behavior is driven by:
  - User input
  - External systems
  - Network or sensor data
- Different events → different system responses

REQUIREMENTS MODELING PRINCIPLE

Principle 4: Partition the Problem

- Break complex problems into smaller parts
- Use **layered or hierarchical models**
- Apply sep**aration of concerns**

Principle 5: Move from Essence to Detail

- Start with **what the user needs**
- Avoid implementation decisions early
- Delay technical choices until design

REQUIREMENTS MODELING PRINCIPLE

Why These Principles Matter

- Applying these principles helps engineers:
  - Understand complex problems
  - Communicate clearly with stakeholders
  - Create models that support good design

REQUIREMENTS ANALYSIS

TYPES OF REQUIREMENTS MODELS

Requirements analysis produces one or more models:

- **Scenario-based models:** User interactions and usage
- **Class-based models:** Objects, attributes, operations
- **Behavioral models:** System reactions to events
- **Data models:** Information domain
- **Flow-oriented models:** Functional transformations

SCENARIO-BASED MODELING

Why scenario-based modeling?

- User satisfaction is a key measure of system success
- Understanding how users interact with a system helps:
  - Clarify requirements
  - Reduce misunderstanding
  - Build meaningful analysis and design models

How it is modeled (UML):

- Use case diagrams
- Activity diagrams
- Sequence diagrams

Scenario-based modeling starts by describing how the system is used

SCENARIO-BASED MODELING

ACTORS AND USER PROFILES

Actors

- Represent roles that interact with the system
- Can be:
  - Human users
  - External systems
  - Hardware devices
- One physical person may play multiple actor roles

User Profiles

- Extend models for:
  - Different platforms (e.g., web vs mobile)
  - Different user viewpoints
- Example:
  - System administrator vs end user

Actors describe who interacts with the system, not how it is implemented

SCENARIO-BASED MODELING

CREATING USE CASES

From user stories to use cases

- User stories: informal, stakeholder language
- Use cases: precise, structured descriptions

Use case definition

- A use case is a “contract for behavior”
- Describes how an actor uses the system to achieve a goal
- Captures interactions between:
  - Actor
  - System

Key questions when writing use cases

- What should be written?
- How much detail is needed?
- How should it be organized?

SCENARIO-BASED MODELING

WHERE USE CASES COME FROM

Use cases are derived from:

- Inception and elicitation activities
- Stakeholder discussions
- Defined system goals
- Identified functional requirements

How to start

- Choose an actor
- List the activities the actor performs
- Focus on **what the system does**, not how

CASE STUDY SCENARIO-BASED MODELING

DEVELOPING ANOTHER PRELIMINARY USE CASE: SAFEHOME

The scene: A meeting room, during the second requirements gathering meeting.

The players: Jamie Lazar, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:
Facilitator: It's time that we begin talking about the SafeHome surveillance function. Let's develop a user scenario for access to the surveillance function.
Jamie: Who plays the role of the actor on this?
Facilitator: I think Meredith (a marketing person) has been working on that functionality. Why don't you play the role?
Meredith: You want to do it the same way we did it last time, right?
Facilitator: Right . . . same way.
Meredith: Well, obviously the reason for surveillance is to allow the homeowner to check out the house while he or she is away, to record and play back video that is captured . . . that sort of thing.

CASE STUDY SCENARIO-BASED MODELING

DEVELOPING ANOTHER PRELIMINARY USE CASE : SAFEHOME

Ed: Will we use compression to store the video?
Facilitator: Good question, Ed, but let's postpone implementation issues for now. Meredith?
Meredith: Okay, so basically there are two parts to the surveillance function . . . the first configures the system including laying out a floor plan—we need to have the AR/VR tools to help the homeowner do this—and the second part is the actual surveillance function itself. Since the layout is part of the configuration activity, I'll focus on the surveillance function.
Facilitator (smiling): Took the words right out of my mouth.
Meredith: Um . . . I want to gain access to the surveillance function either via a mobile device or via the Internet. My feeling is that the Internet access would be more frequently used. Anyway, I want to be able to display camera views on a mobile device or PC and control pan and zoom for a specific camera. I specify the camera by selecting it from the house floor plan. I want to selectively record camera output and replay camera output. I also want to be able to block access to one or more cameras with a specific password. I also want the option of seeing small windows that show views from all cameras and then be able to pick the one I want enlarged.

CASE STUDY SCENARIO-BASED MODELING

DEVELOPING ANOTHER PRELIMINARY USE CASE : SAFEHOME

Jamie: Those are called thumbnail views.
Meredith: Okay, then I want thumbnail views of all the cameras. I also want the interface for the surveillance function to have the same look and feel as all other SafeHome interfaces. I want it to be intuitive, meaning I don't want to have to read a manual to use it.
Facilitator: Good job. Now, let's go into this function in a bit more detail . . .

CASE STUDY SCENARIO-BASED MODELING

DOCUMENTING USE CASE

From Informal to Formal Use Cases

- Informal use cases are often sufficient
- Formal use cases are needed when:
  - The functionality is critical
  - The interaction is complex
  - Many exceptions exist

Why formalize?

- Clarity
- Precision
- Reviewability

CASE STUDY SCENARIO-BASED MODELING

DOCUMENTING USE CASE

A formal use case typically includes:

- **Goal in context:** overall purpose
- **Precondition:** what must be true before starting
- **Trigger:** what initiates the use case
- **Main scenario:** normal interaction steps
- **Exceptions:** alternative or error paths

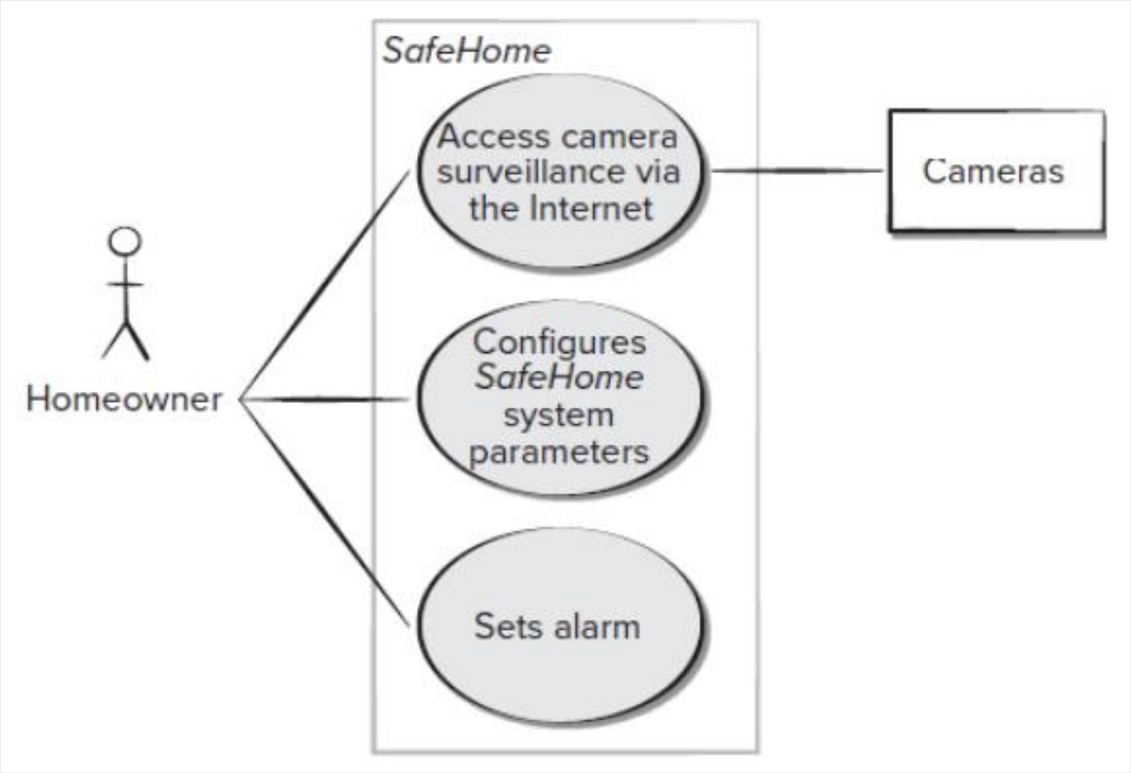
Purpose

Refines informal scenarios into precise behavior descriptions

This structure turns conversations into something engineers can work with.

CASE STUDY SCENARIO-BASED MODELING

USE CASE DIAGRAM: SAFEHOME



What the diagram shows

- Actor: **Homeowner**
- System boundary: **SafeHome**
- Use cases:
  - Access camera surveillance
  - Configure system parameters
  - Set alarm

What it does NOT show

- Internal logic
- Sequence of steps
- Non-functional requirements

Diagrams summarize what the system offers, not how it works

CLASS-BASED MODELING

IDENTIFYING ANALYSIS CLASSES

How to identify analysis classes

- Start from **usage scenarios / use cases**
- Perform a **grammatical parse:**
  - Underline nouns / noun phrases
  - Record them in a table
  - Note synonyms

Class relevance

- **Solution space:** required to implement the system
- **Problem space:** helps describe the problem only

CLASS-BASED MODELING

IDENTIFYING ANALYSIS CLASSES

Common forms of analysis classes

- External entities (people, devices, other systems)
- Things in the information domain (reports, displays, signals)
- Events or occurrences during system operation
- Roles played by users
- Organizational units
- Places relevant to the system context
- Physical or logical structures (sensors, devices, computers)

IDENTIFYING ANALYSIS CLASSES

Refining Potential Analysis Classes

From nouns to potential classes

- All nouns from the narrative are listed as potential classes
- Each potential class must be evaluated
- Not all potential classes will become analysis classes

CLASS-BASED MODELING

IDENTIFYING ANALYSIS CLASSES

Selection criteria for analysis classes (Coad & Yourdon)

A potential class should be included if it has:

- **Retained information:** The system must remember information about it
- **Needed services:** It has meaningful operations that change its state
- **Multiple attributes:** More than a single simple attribute
- **Common attributes:** Attributes apply to all instances of the class
- **Common operations:** Operations apply consistently to all instances
- **Essential requirements:** External entities that produce or consume essential information

CLASS-BASED MODELING

IDENTIFYING ANALYSIS CLASSES: CASE STUDY SAFEHOME

Extracting the nouns, we can propose several potential classes:

Potential Class	General Classification
homeowner	role or external entity
sensor	external entity
control panel	external entity
installation	occurrence
system (alias security system)	thing
number, type	not objects, attributes of sensor
master password	thing
telephone number	thing
sensor event	occurrence
audible alarm	external entity
monitoring service	organizational unit or external entity

CLASS-BASED MODELING

IDENTIFYING ANALYSIS CLASSES: EVALUATING POTENTIAL CLASSES (SAFEHOME EXAMPLE)

- Each potential class is evaluated against the selection characteristics.
- Rejection does not mean "unimportant"
- Some rejected classes become:
  - Attributes of accepted classes
    - number, type → attributes of Sensor
    - master password, telephone number → attributes of System
- Decisions are not final: Classes may be revised later as understanding improves

Potential Class	Characteristic Number That Applies
homeowner	rejected: 1, 2 fail even though 6 applies
sensor	accepted: all apply
control panel	accepted: all apply
installation	rejected
system (alias security function)	accepted: all apply
number, type	rejected: 3 fails, attributes of sensor
master password	rejected: 3 fails
telephone number	rejected: 3 fails
sensor event	accepted: all apply
audible alarm	accepted: 2, 3, 4, 5, 6 apply
monitoring service	rejected: 1, 2 fail even though 6 applies

CLASS-BASED MODELING

DEFINING CLASS ATTRIBUTES

What are attributes?

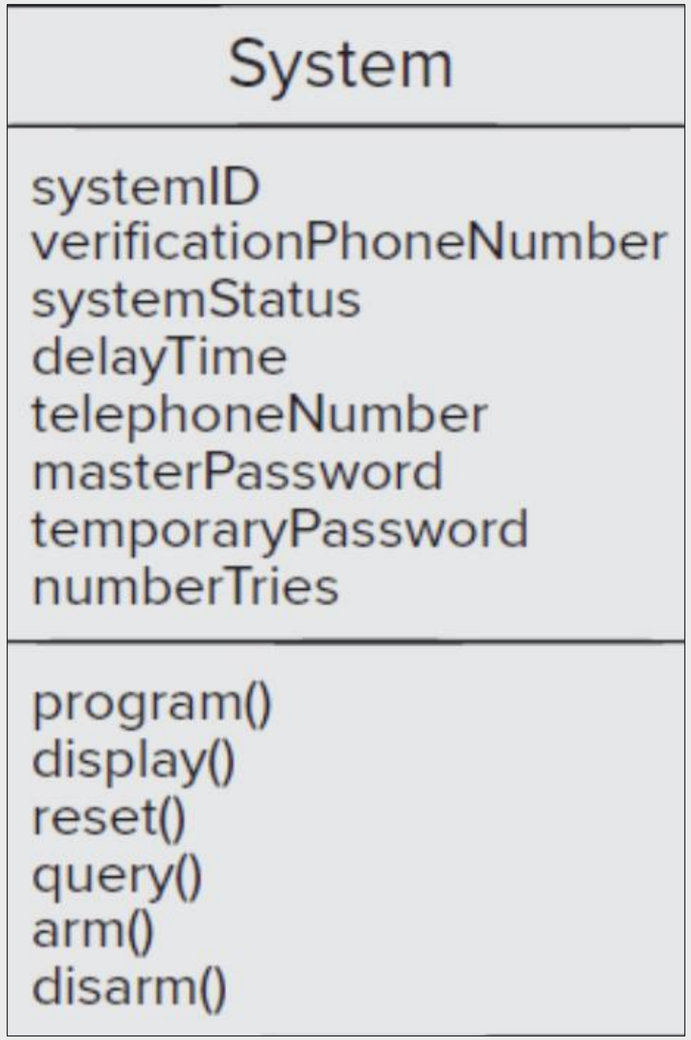
- Attributes describe the state of a class
- They clarify what the class means in the problem

Example: System class (SafeHome)

Composite attributes:

- Identification information
  - systemId
  - verificationPhoneNumber
  - systemStatus
- Alarm response information
  - delayTime
  - telephoneNumber
- Activation / deactivation information
  - masterPassword
  - temporaryPassword
  - numberTries

Design note: Items associated multiple times (e.g., Sensors) are modeled as separate classes, not attributes



CLASS-BASED MODELING

DEFINING CLASS ATTRIBUTES

What are operations?

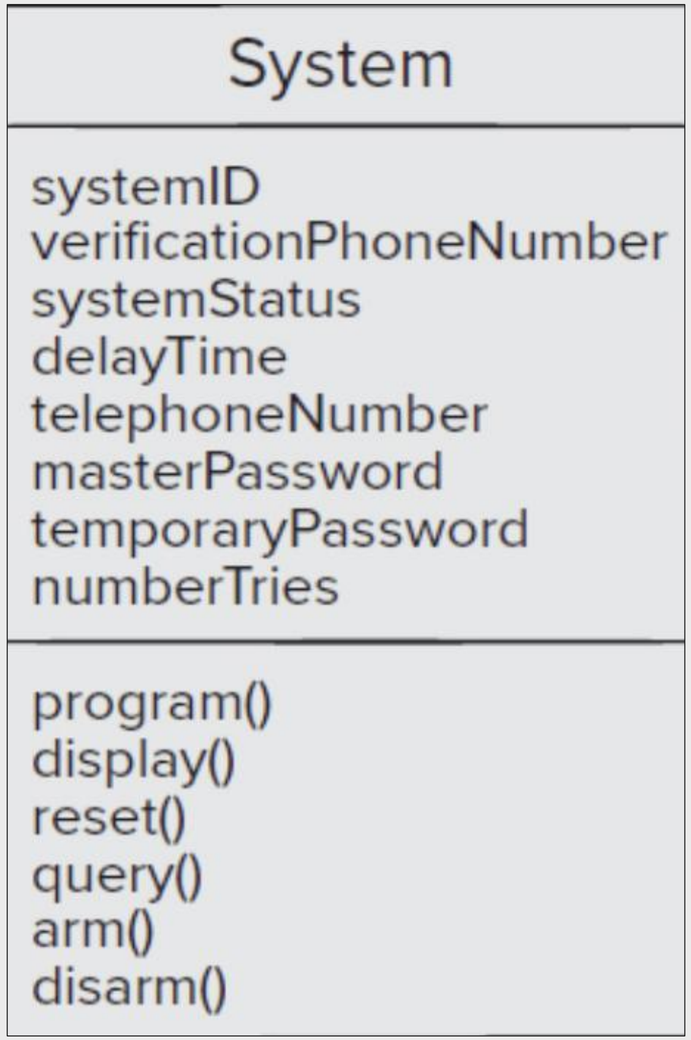
- Operations define the behavior of a class
- They act on:
  - Attributes
  - Associations with other classes

Common operation categories

- Data manipulation (add, delete, update)
- Computation
- State inquiry
- Event monitoring

Example: System class operations

- program()
- display()
- reset()
- query()
- arm()
- disarm()



CLASS-BASED MODELING

UML CLASS MODEL

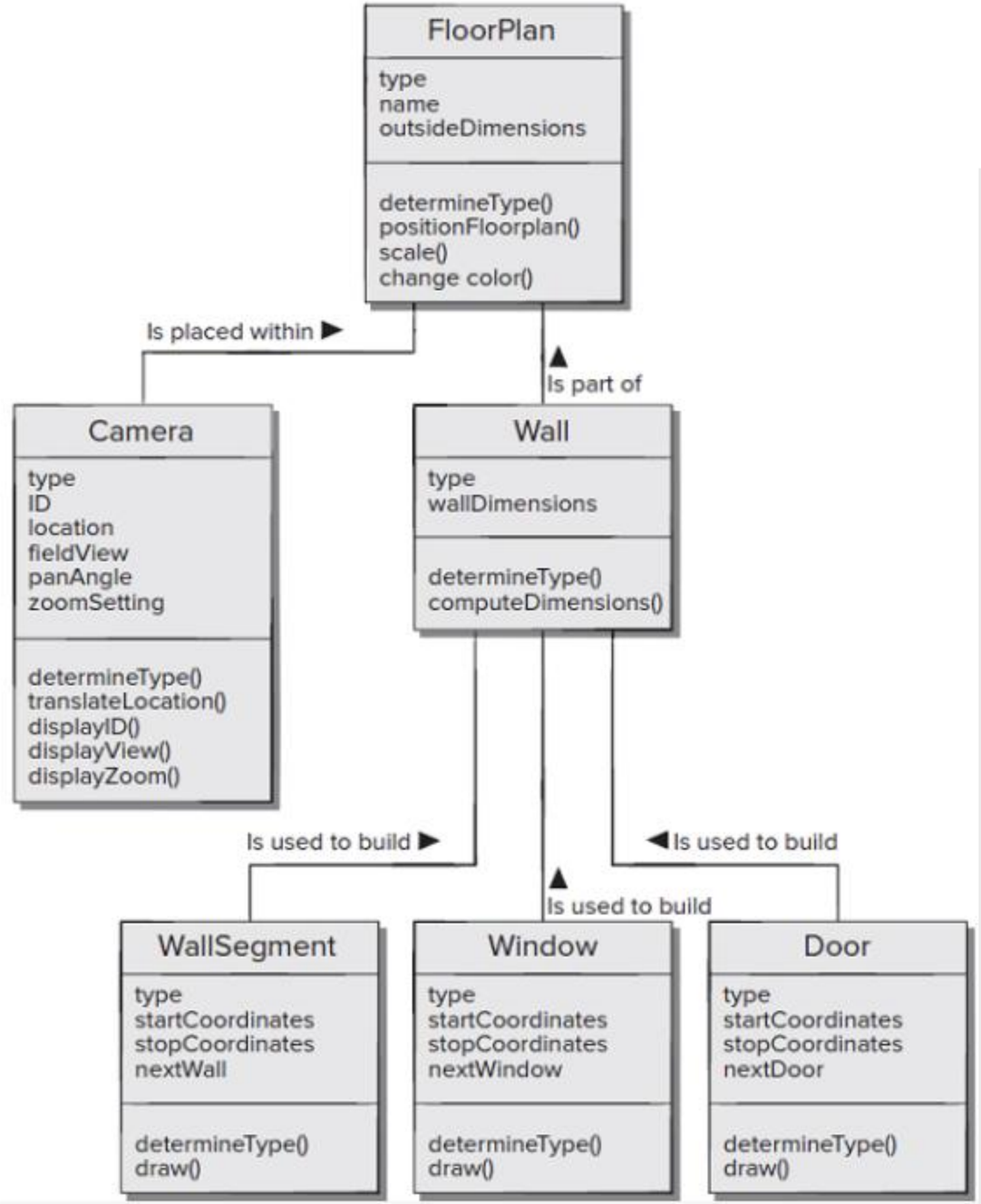
Purpose: Represents the structural relationships among classes in the SafeHome floor-plan and camera subsystem.

Key Relationship to Notice

The FloorPlan class is defined by identifying a set of associations between FloorPlan and two other classes, Camera and Wall. The class Wall is associated with three classes that allow a wall to be constructed, WallSegment, Window, and Door.

Design Decisions Reflected in the Model

- Physical objects are modeled as classes
- Repeated structures are separated into reusable components
- Geometry and display behavior are encapsulated in each class



FUNCTIONAL MODELING

What functional modeling addresses

- Describes what processing the system performs
- Focuses on behavior, not structure

Two levels of functional abstraction

1. **User-observable functionality**
   - Functions initiated directly by users
   - What the user sees as system behavior
   - Example: calculating a mortgage payment
2. **Class-level operations**
   - Operations inside analysis classes
   - Manipulate attributes and support collaboration between classes
   - Implement user-visible functions internally

Functional modeling connects what users experience with how the system performs it internally.

FUNCTIONAL MODELING

A PROCEDURAL VIEW

Procedural view of functionality

- Describes how processing flows inside the system
- Focuses on steps, decisions, and parallel activities
- Independent of implementation details

UML Activity Diagram

- Used to represent processing logic graphically
- Supplements use cases by showing:
    - Flow of actions
    - Decision points
    - Possible parallel behavior

Analysis-level guideline

- Use activity diagrams only when functionality is complex
- Avoid over-modeling simple interactions

FUNCTIONAL MODELING

A PROCEDURAL VIEW

Example: takeControlOfCamera() operation

What the diagram shows

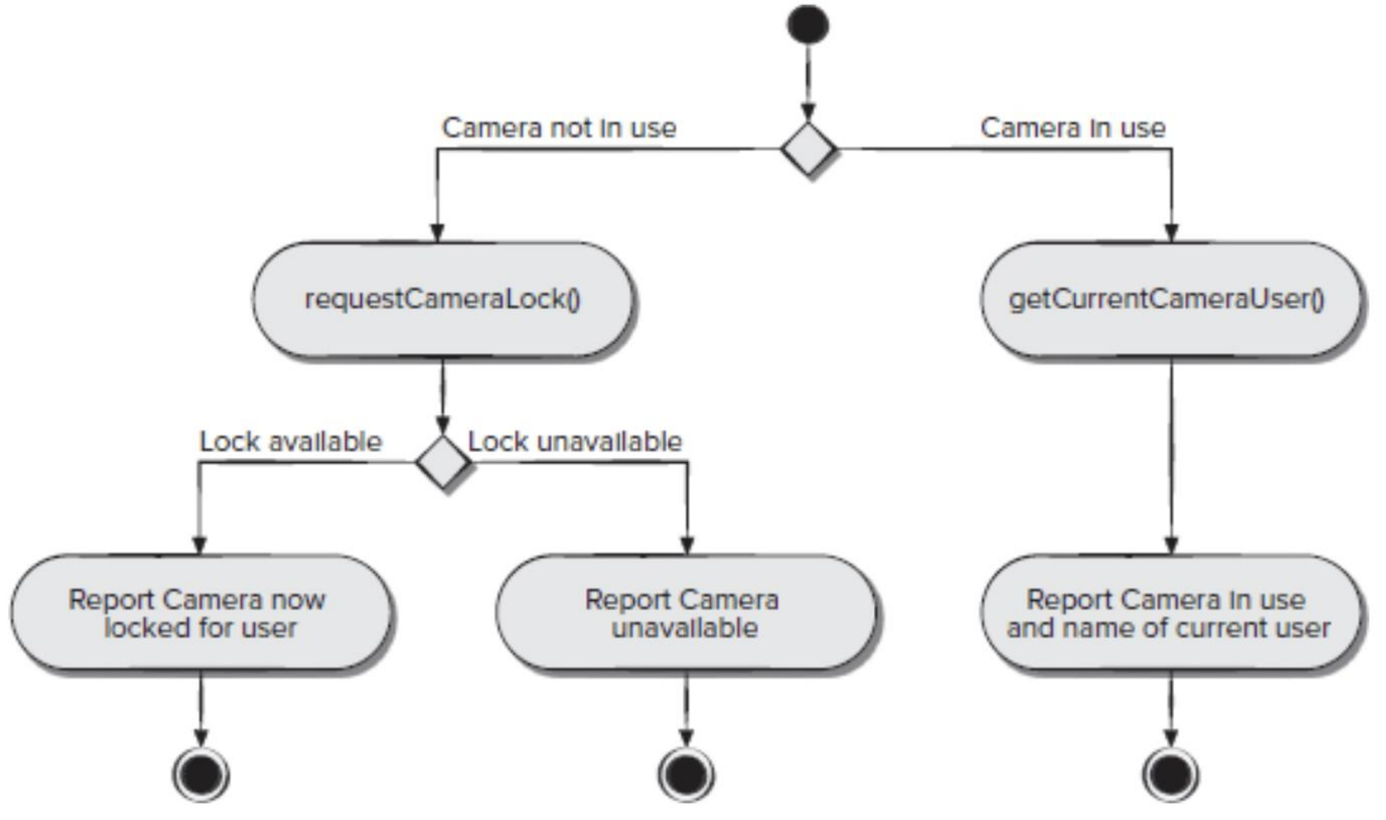
- Start and end of processing
- Decision: camera in use or not
- Conditional logic:
  - Request camera lock
  - Handle lock availability
  - Report current camera user if locked

Supporting operations

- requestCameraLock()
- getCurrentCameraUser()

What is intentionally excluded

- Communication protocols
- Interface details
- Implementation logic



FUNCTIONAL MODELING

UML SEQUENCE DIAGRAMS

Purpose of sequence diagrams

- Used for behavioral modeling
- Show how events trigger interactions between objects over time
- Derived directly from use cases

What a sequence diagram represents

- Objects involved in a scenario
- Events/messages exchanged between objects
- Order of interactions as time progresses

A **sequence diagram is a behavioral view of a use case.**

FUNCTIONAL MODELING

UML SEQUENCE DIAGRAMS

How to read the diagram

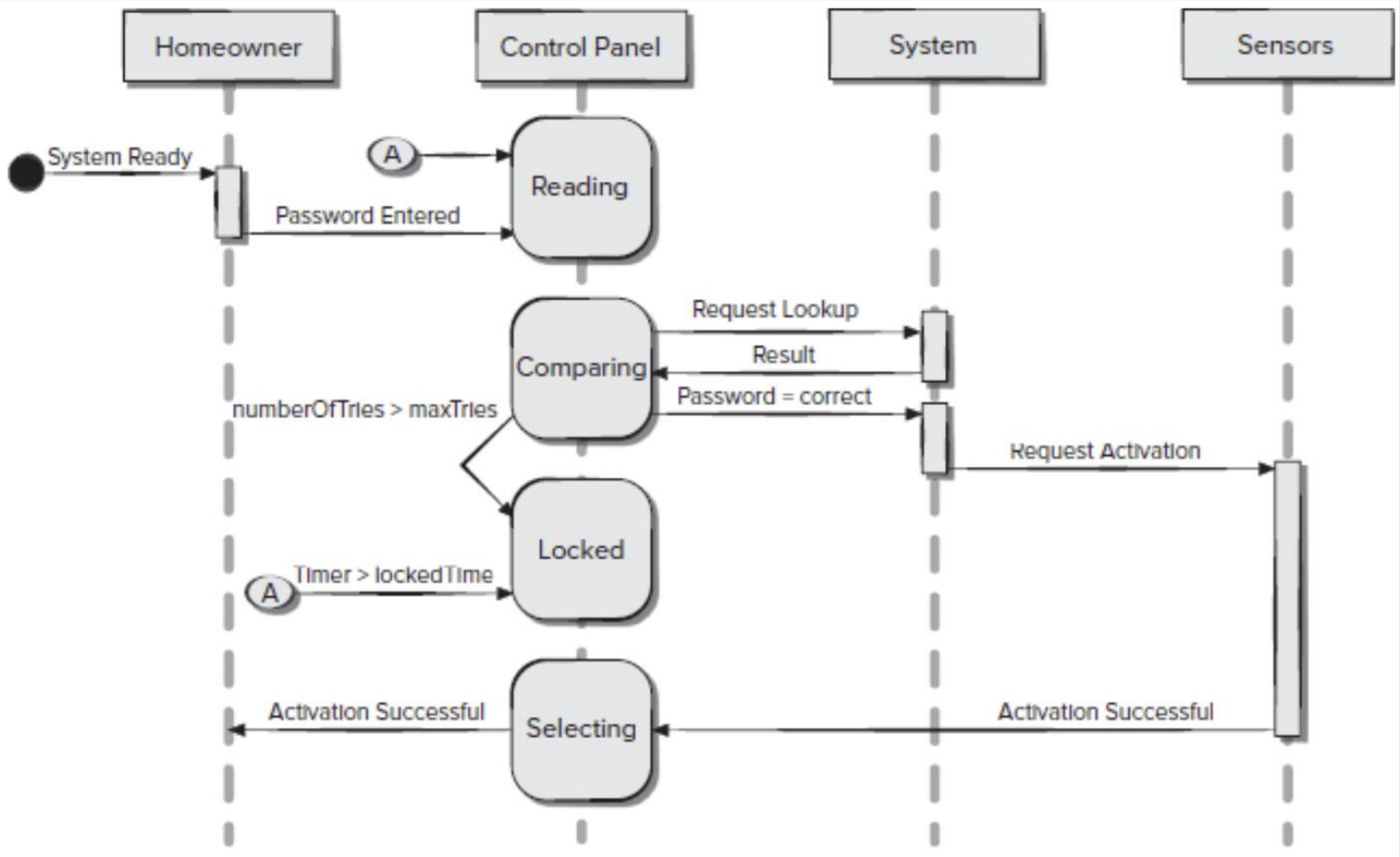
- Vertical axis: **time flows downward**
- Horizontal axis: **objects / classes**
- Arrows: **events or messages**
- Narrow vertical rectangles: **processing activity**
- States may appear along the timeline

SafeHome example (partial)

- External event: system ready
- Homeowner enters password
- Control panel requests password lookup
- System validates password
- Sensors are activated
- Success is reported back to the homeowner

Modeling focus

- Shows how behavior flows between objects
- Omits:
  - Implementation details
  - Database structure
  - Communication protocols



BEHAVIORAL MODELING

What behavioral modeling describes

- How software **responds to events or stimuli**
- Focuses on **system behavior over time**
- Useful for validating system logic before design

UML diagrams used

- **Activity diagrams** → response to internal events
- **State diagrams** → response to external events

How the behavioral model is created

1. Review all use cases to understand interaction sequences
2. Identify events that drive system behavior
3. Define event sequences for each use case
4. Build state diagrams for the system
5. Review the model for accuracy and consistency

BEHAVIORAL MODELING

IDENTIFYING EVENTS FROM USE CASES

What is an event?

- An **event occurs when an actor and the system exchange information**
- The event is **the exchange itself**, not the data value

How to identify events

- Examine use cases for **points of interaction**
- Look for moments where:
  - An actor sends information to the system
  - The system responds or changes behavior

Example: SafeHome security use case

- Homeowner enters password → **event: password entered**
- System compares password → **event: password compared**
- Results of comparison affect control flow

Important distinctions

- Some events **trigger control flow changes**
- Others only provide information
- Events are assigned to responsible objects:
  - Actors may **generate** events
  - System objects may **recognize** events

BEHAVIORAL MODELING

UML STATE DIAGRAMS

What a state diagram models

- How an object changes state in response to events
- Focuses on behavior over time

Two views of state

- Passive state
  - Current values of object attributes
- Active state
  - The object's current mode of operation
  - Indicates ongoing processing or activity

State transitions

- Triggered by events
- Events cause objects to move from one active state to another

State diagrams describe the life history of an object.

BEHAVIORAL MODELING

UML STATE DIAGRAMS

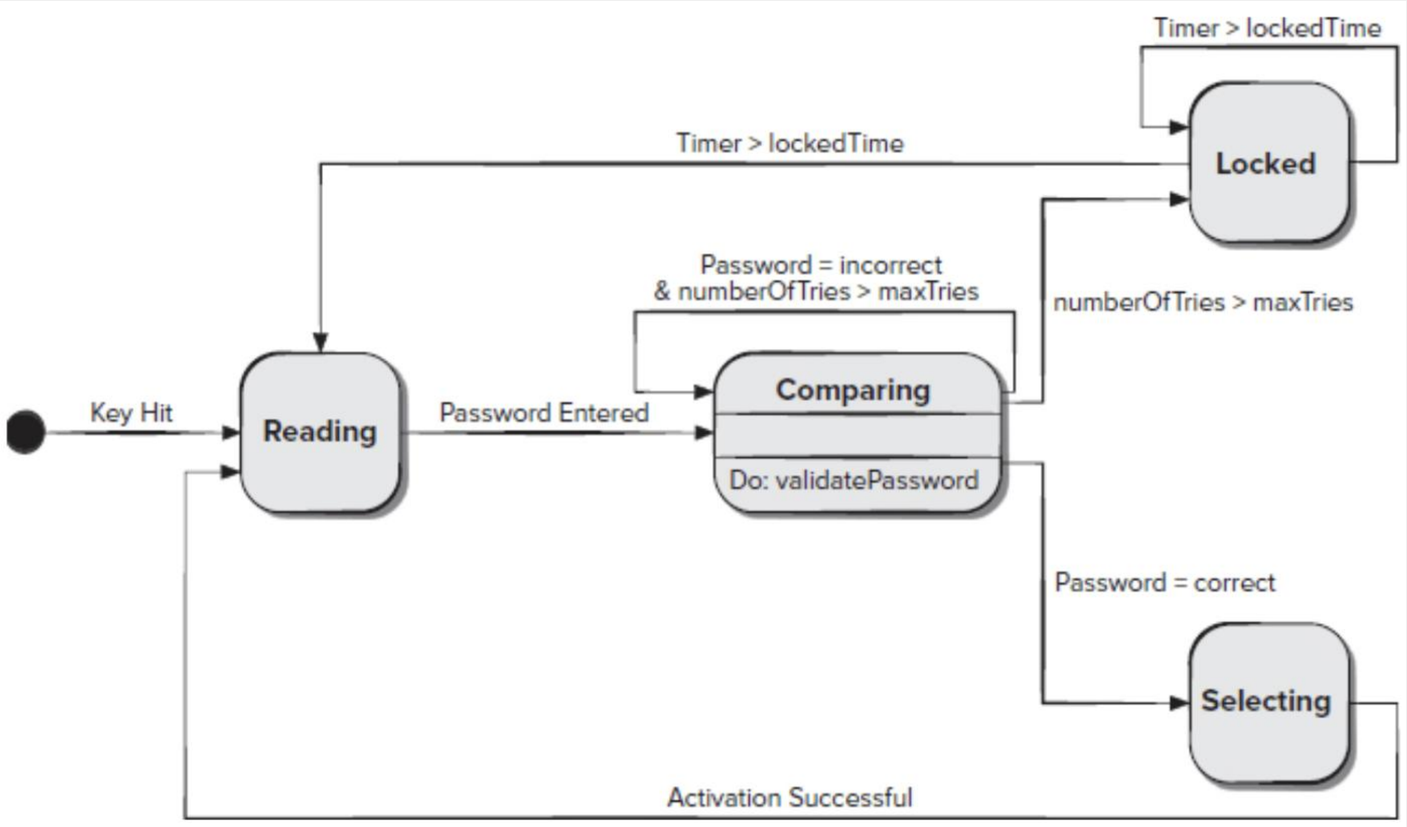
Example: ControlPanel class (SafeHome)

What the diagram shows

- Active states: Reading, Comparing, Locked and Selecting
- Transitions between states driven by events

Transition elements

- Event (trigger)
  - e.g., password entered
- Guard condition
  - Boolean expression controlling the transition
  - Example: if (password input = 4 digits)
- Action
  - Operation executed during the transition
  - Example: validatePassword()



Important modeling rule

- Guards depend on attribute values (passive state)
- Actions invoke class operations

BEHAVIORAL MODELING

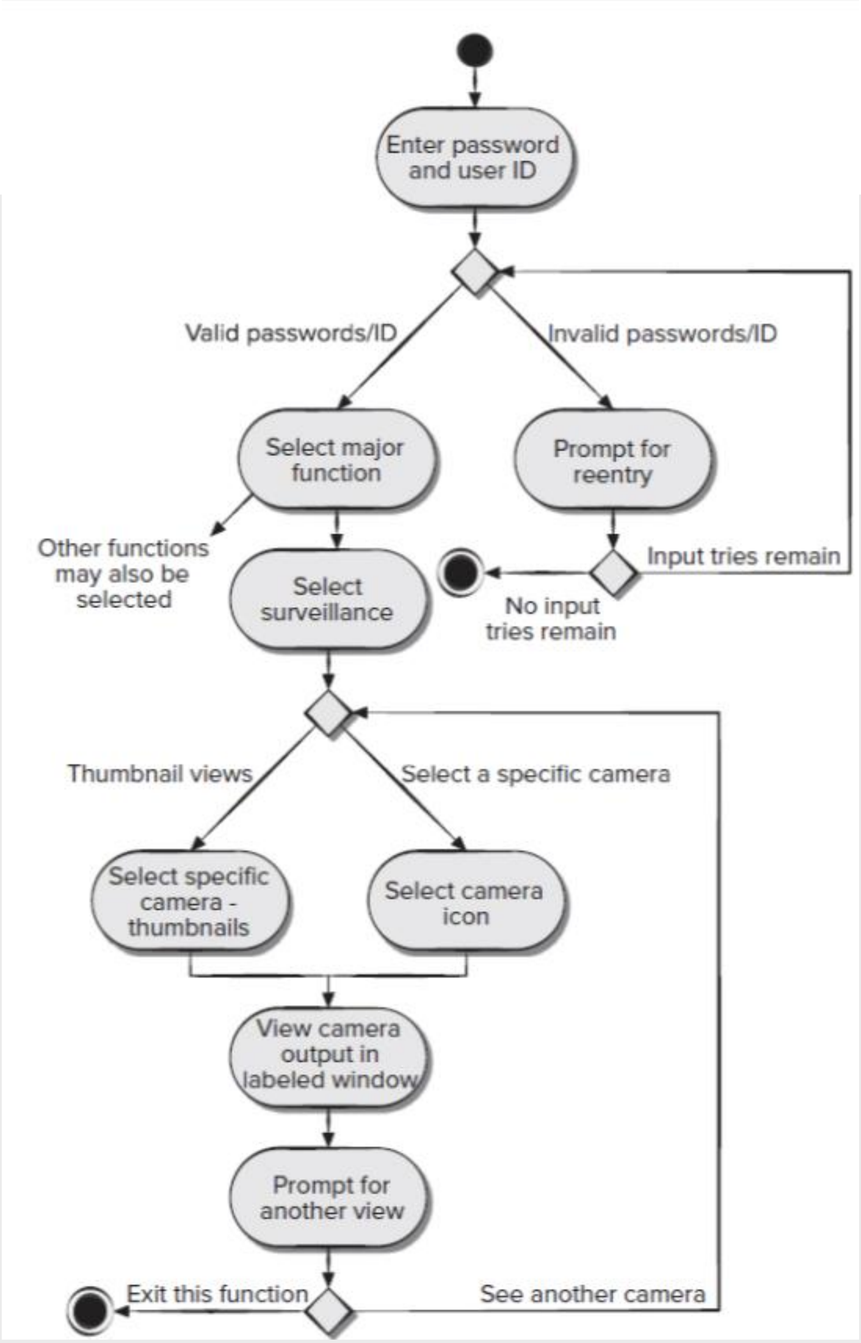
UML ACTIVITY DIAGRAMS

Purpose of an activity diagram

- Supplements a use case with a visual flow of actions
- Shows how the system **reacts to internal events**
- Highlights:
  - Activities
  - Decisions
  - Alternative paths

Key characteristics

- Rounded rectangles → activities
- Diamonds → decisions
- Arrows → control flow
- May include details implied by the use case(e.g., limited password attempts)



BEHAVIORAL MODELING

SWIMLANE ACTIVITY DIAGRAMS

What is a swimlane diagram?

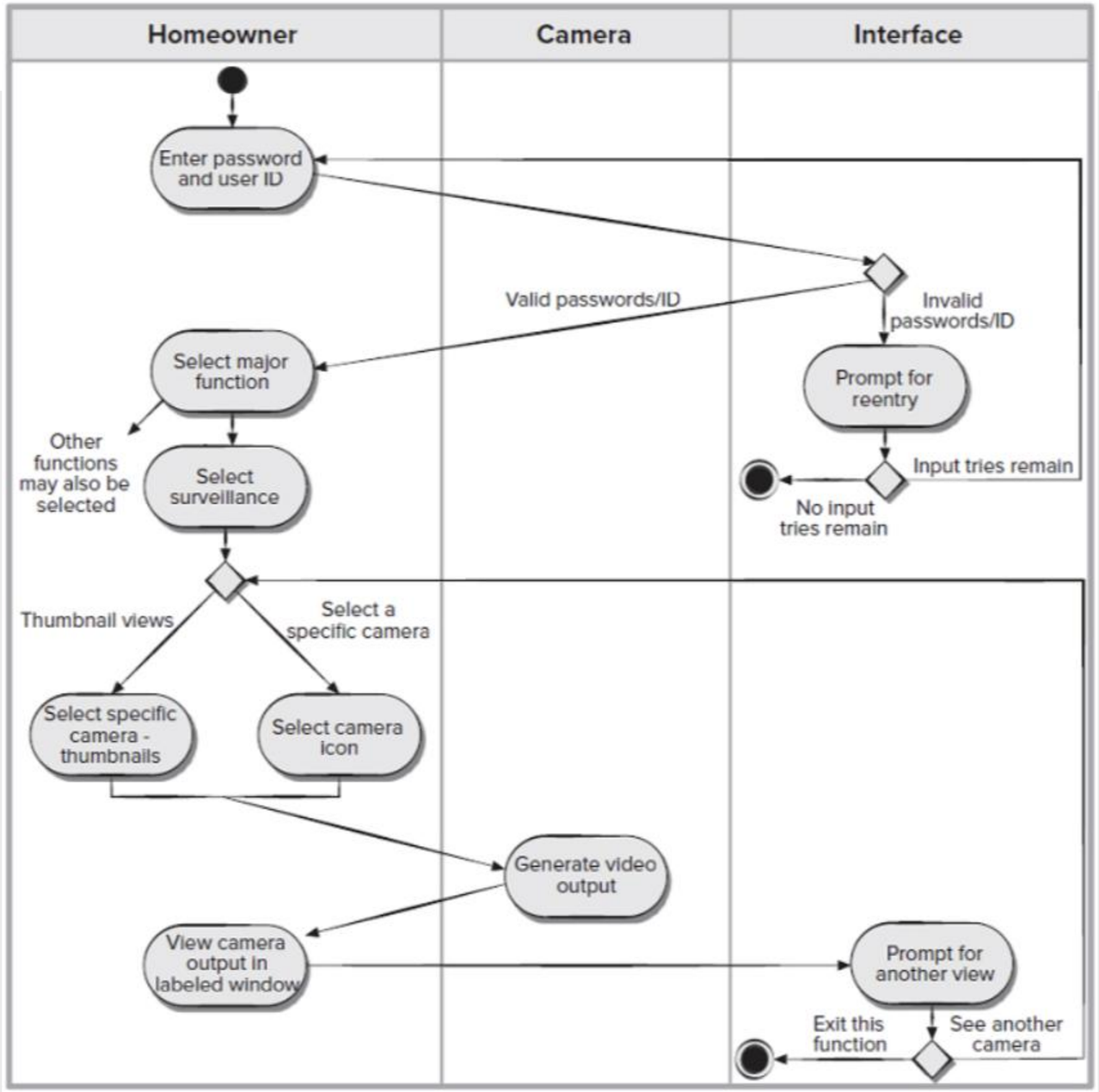
- A variation of the activity diagram
- Shows **who is responsible** for each activity

How swimlanes work

- Vertical lanes represent:
  - Actors, or
  - Analysis classes
- Activities are placed in the lane of the responsible entity

SafeHome example

- Lanes represent:
  - **Homeowner**
  - **Camera**
  - **Interface**
- Interface responsibilities include:
  - Prompt for reentry
  - Prompt for another view
- Control flows across lanes to show collaboration



People
Innovation
Excellence

REFERENCES

- Roger Pressman and Bruce Maxim. (2020). Software Engineering: A Practitioner's Approach. 9th. McGraw Hill. New York. ISBN: 978-1259872976.